

QIDK PROJECT 10 · EMBEDDED SYSTEMS WORKSHOP

Evaluation of Image Inpainting Algorithms and Their Optimization on the QIDK Platform

On-device GAN and diffusion-based inpainting, benchmarked and optimized for Snapdragon NPU/GPU/CPU

TEAM 19 — BLACK & WHITE

Aryan Kulkarni · Ashwin Aravind · Tarsh Singh · Raghavendra Navneet Nawander

Mentor / TA: Vedant · Faculty Advisor: Dr. Deepak Gangadharan, IIIT Hyderabad

Progress checkpoint — September 2026

THE PROBLEM

Why image inpainting, and why on-device?

Image inpainting fills in missing, damaged, or masked regions of an image so the result looks natural — used for object removal, photo restoration, and generative photo editing.

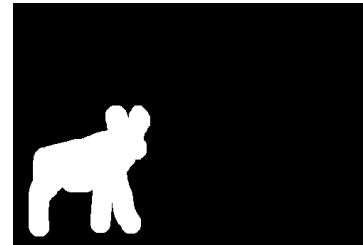
Running these models on a phone (QIDK / Snapdragon) instead of the cloud means no upload latency, no server cost, and full data privacy — but mobile NPUs are far more constrained than a GPU workstation.

Our brief (Project 10)

Evaluate multiple image-inpainting algorithms and optimize their deployment on the QIDK — comparing accuracy, speed, and hardware efficiency across CPU / GPU / NPU.



input image



mask (region to remove)



inpainted output

A running example from our own benchmark set — the antelope is removed and the background is regenerated by the model.

What is the QIDK?

The Qualcomm Innovators Development Kit (QIDK) is a Snapdragon-powered device used to develop and benchmark on-device AI. It exposes four distinct compute engines, each with a different speed / power / precision trade-off:



CPU

General purpose. Slowest, most flexible, easiest to debug — our first working target for every model.



GPU

Good throughput for dense matrix ops. Used as a mid-point sanity check before NPU deployment.



HTP / DSP (NPU)

Qualcomm's Hexagon Tensor Processor — the fast, low-power engine phones actually ship AI on. Needs INT8-quantized models.



AIP

AI accelerator path exposed by SNPE for select model graphs; explored as an alternate NPU route.

Note: on Qualcomm chips, “DSP” and “HTP” refer to the same NPU block — both names surfaced in our own team notes.

Four inpainting models, two families

GAN

AOT-GAN

Aggregated-transformation GAN. Fast, single forward pass. Qualcomm-provided reference model, trained on faces.

researchmm/AOT-GAN-for-Inpainting

GAN

LaMa-Dilated

Fourier-convolution GAN good at large masks / repetitive textures. Pulled as a ready .dlc from Qualcomm AI Hub.

huggingface.co/qualcomm/LaMa-Dilated

GAN

MI-GAN

A GAN baseline built specifically for mobile — small, fast, but needed manual ONNX→DLC compilation ourselves.

picsart-ai-research/mi-gan

DIFFUSION

Stable Diffusion Inpainting

Full text-guided diffusion pipeline: CLIP tokenizer, VAE encode/decode, Euler scheduler, custom-masked latents — built end-to-end by us on QIDK.

Qualcomm AI Hub SD v2.1 + our C++ runner

From a trained model to on-device inference

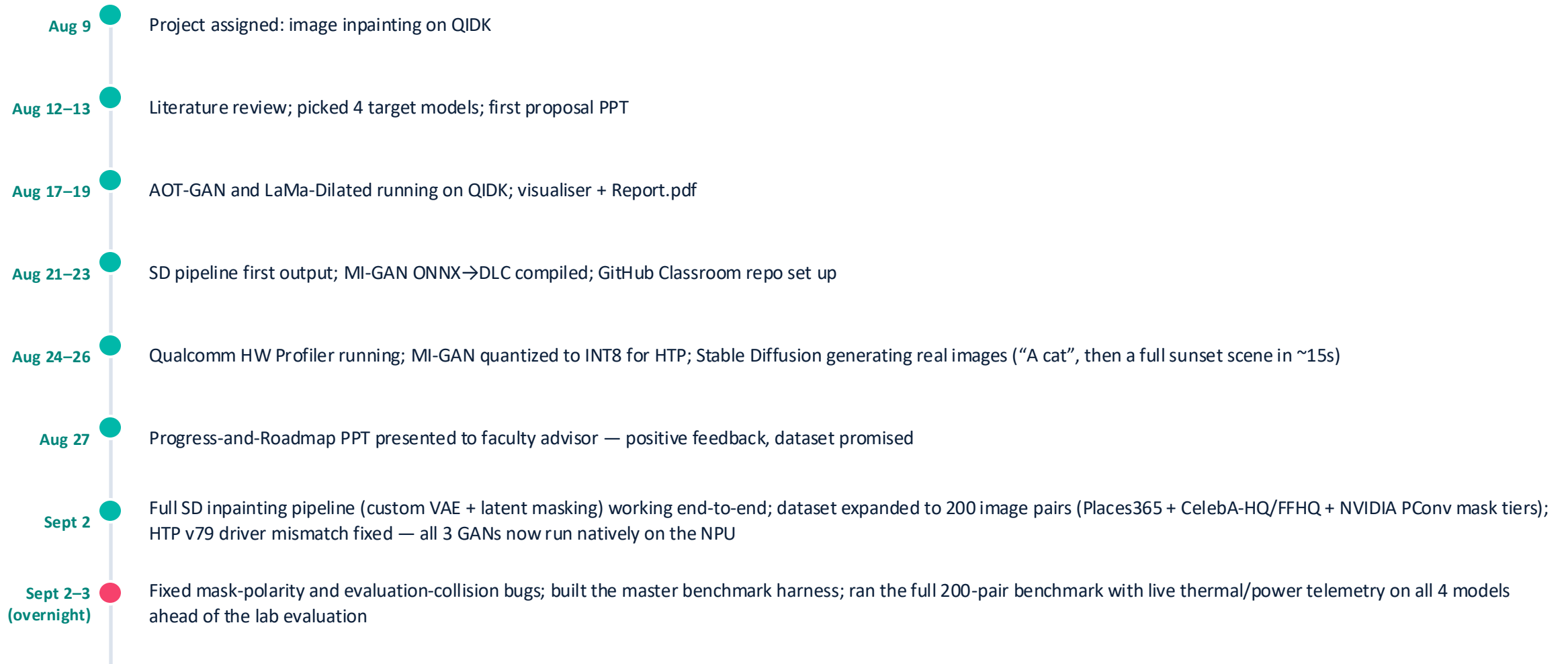


Our repo's numbered scripts mirror this exactly: 01-convert_to_raw.py → 01b-generate_input_list.py → 02-push_to_device.py → 03-run_inference.py → 04-convert_to_img.py

A real snag we hit and fixed

LaMa-Dilated's raw output came back with fewer bytes than expected for float32 — the fix was reinterpreting the buffer as uint8, plus adding explicit transpose ops to the model's input/output before recompiling to .dlc, so it now runs cleanly on CPU, GPU and HTP.

Timeline — assignment to evaluation



What we've done so far



Set up local systems (Ubuntu + QAIRT/SNPE toolchain) to compile and run .dlc models



Built a Stable Diffusion pipeline: custom CLIP-style tokenizer, INT8 VAE encoder, latent masking, Euler scheduler — generating real images on-device



Expanded our benchmark set to 200 image pairs across scenes, faces, and three mask-difficulty tiers



Built a single-command master benchmark harness with live thermal/power telemetry and cooldown control



Ran AOT-GAN, LaMa-Dilated, and our own compiled MI-GAN — all three now execute natively on the Hexagon HTP v79 NPU via --use_dsp



Assembled a complete text-guided image-inpainting pipeline (see the sunset example ahead)



Fixed three real bugs: mask-polarity inversion, evaluation file collisions, and an HTP v2.47/v2.49 driver mismatch



Ran the full quantitative benchmark — PSNR, SSIM, and LPIPS — across all four models on real hardware

4

models, all running natively on NPU

200

image pairs in our benchmark set

0.62 J

per repair — MI-GAN, our efficiency champion

28.3 dB

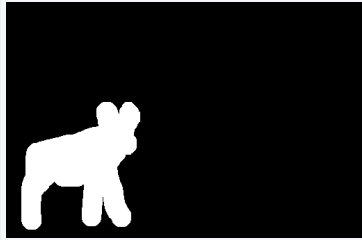
best global PSNR — AOT-GAN

RESULTS

GAN inpainting — real outputs from our pipeline



Input



Mask



Output

Wildlife scene: the masked antelope is removed and the plains background is regenerated to match lighting and grass texture.



Input



Mask



Output

Harder case: a moving subject against a dynamic, textured background (breaking waves) — a good stress test for the GAN models.

Stable Diffusion, generating real images on QIDK



"A cat"

First successful on-device generation (24 Aug)



"A cinematic shot of sunset"

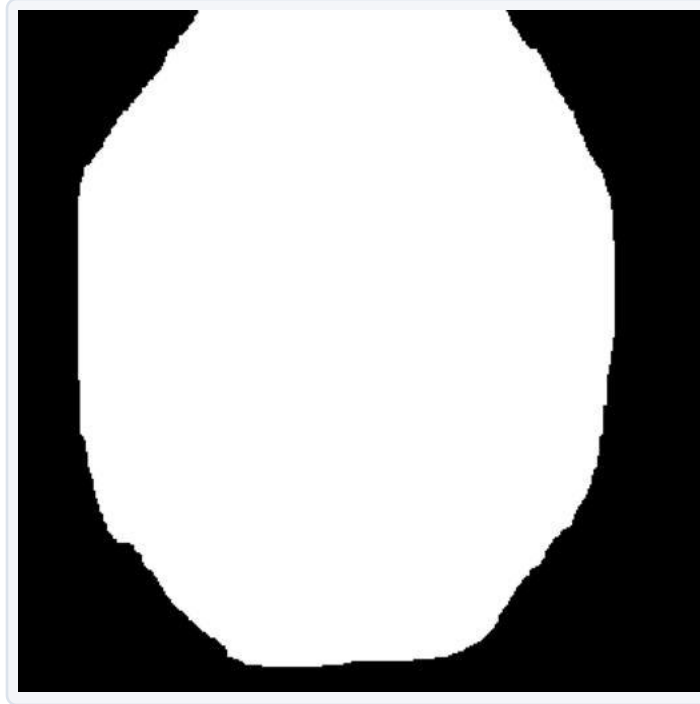
Full pipeline, guidance 2.0, ~15s (26 Aug)

These are unedited, real outputs from our own C++ inference runner on the QIDK — not reference images.

Text-guided inpainting, end to end



Input photo



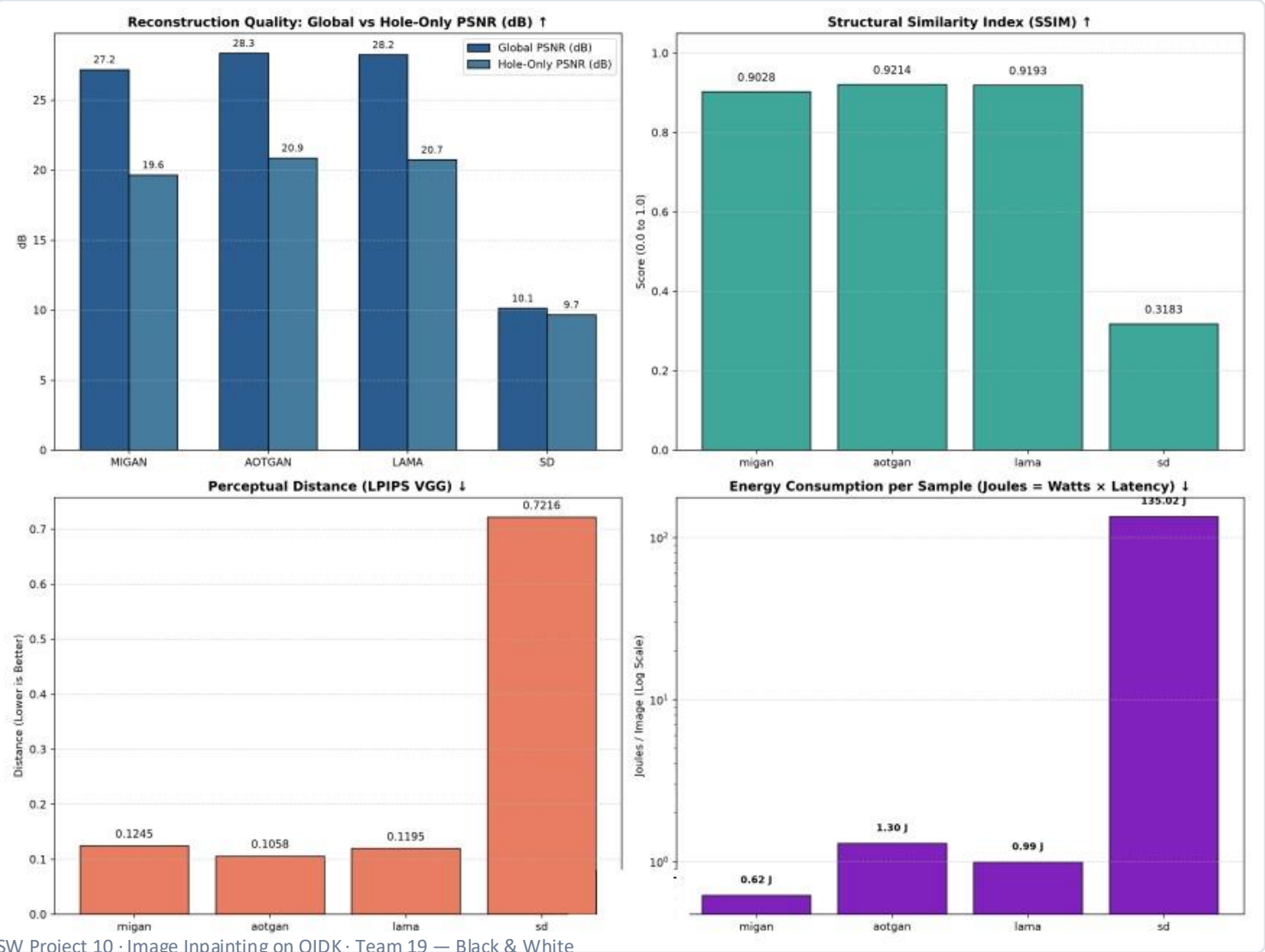
Mask



Prompt: "a cinematic shot of sunset"

The masked region of a real photo (a roadside stop sign) is regenerated according to the text prompt, while everything outside the mask is preserved from the original image — full pipeline, guidance scale 2.0, on-device.

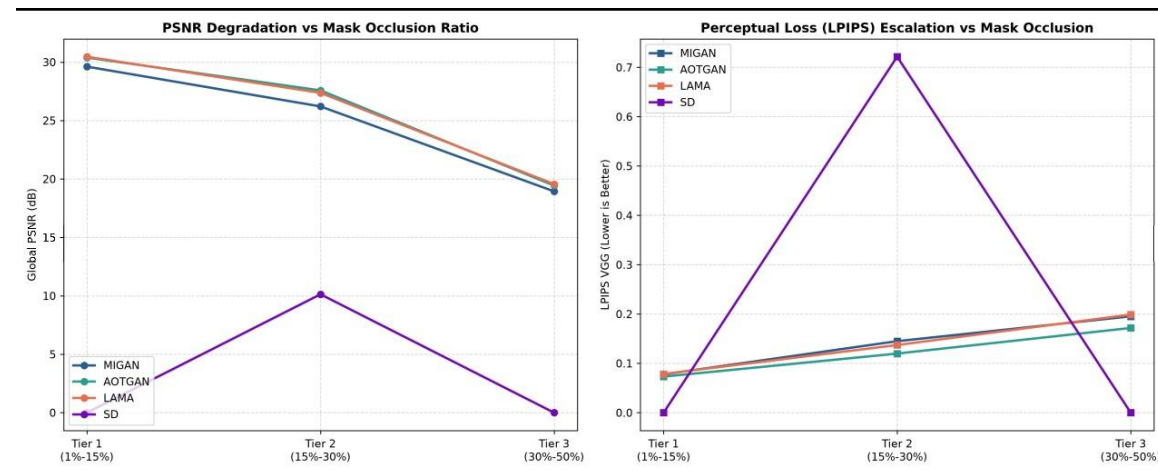
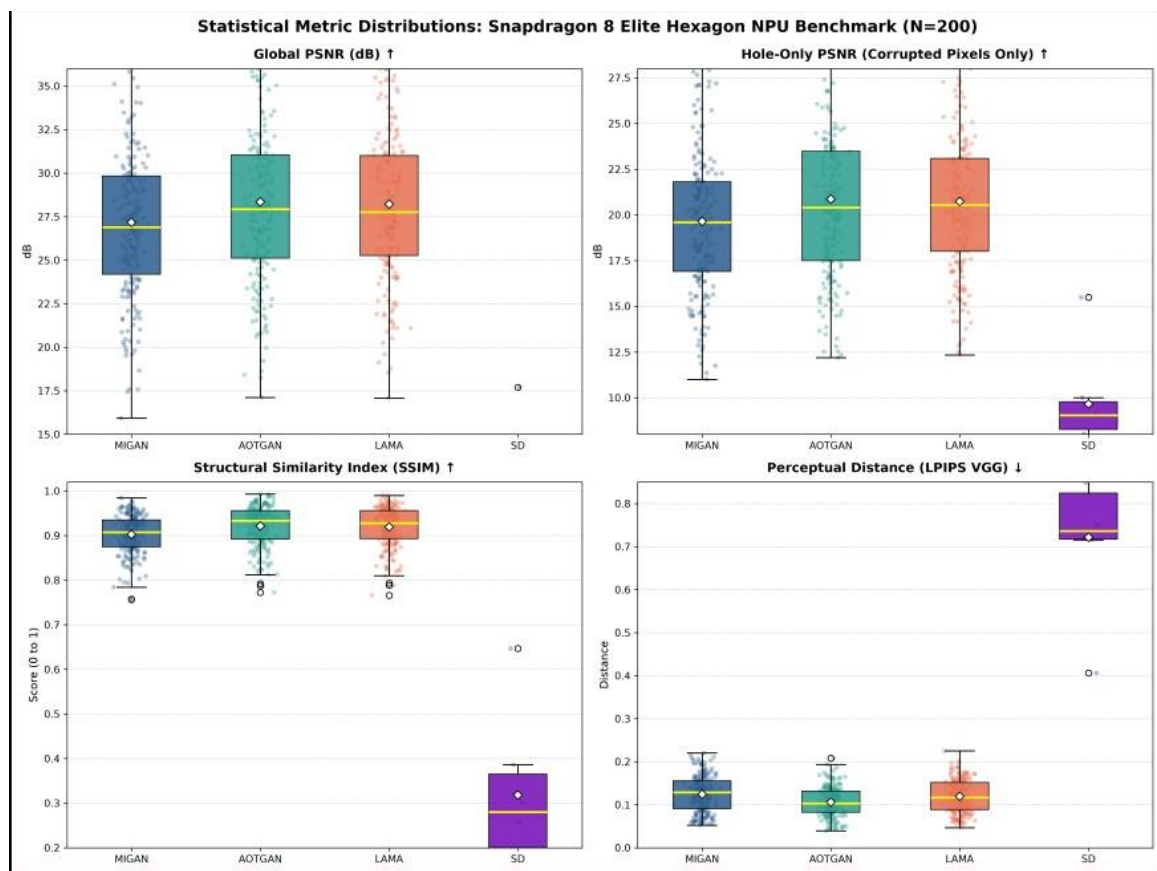
The full benchmark — 200 image pairs, on-device



Model	PSNR	SSIM	Energy
MIGAN	27.2 dB	0.903	0.62 J
AOT-GAN	28.3 dB	0.921	1.30 J
LaMa	28.2 dB	0.919	0.99 J
Stable Diff.	10.1 dB*	0.318*	135.0 J

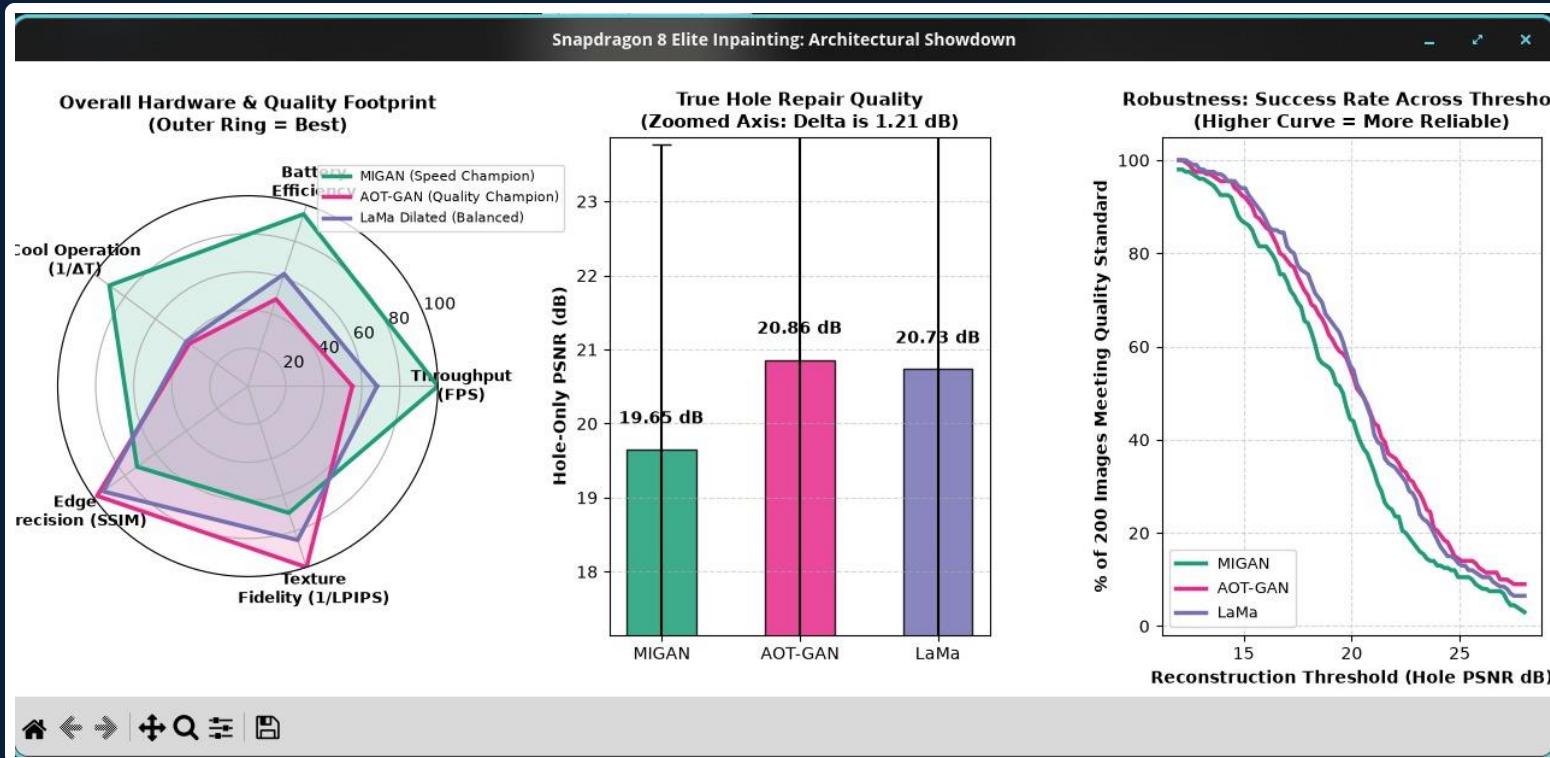
**Diffusion models hallucinate novel, plausible content rather than matching exact ground-truth pixels — pixel-alignment metrics penalize this by design, and this SD checkpoint isn't fine-tuned for inpainting yet.*

Distributions (N=200) and stress under harder masks



Left: full statistical spread (N=200) — boxes show the real variance behind each headline average, not just a single number. Right: as mask coverage grows past ~30% of the image, all models degrade sharply — LaMa holds up best in extreme cases thanks to its wide Fourier receptive field.

The architectural showdown



MI-GAN — efficiency champion

0.62 J/repair, only +9.6°C thermal rise. Ideal for a continuous mobile camera pipeline.

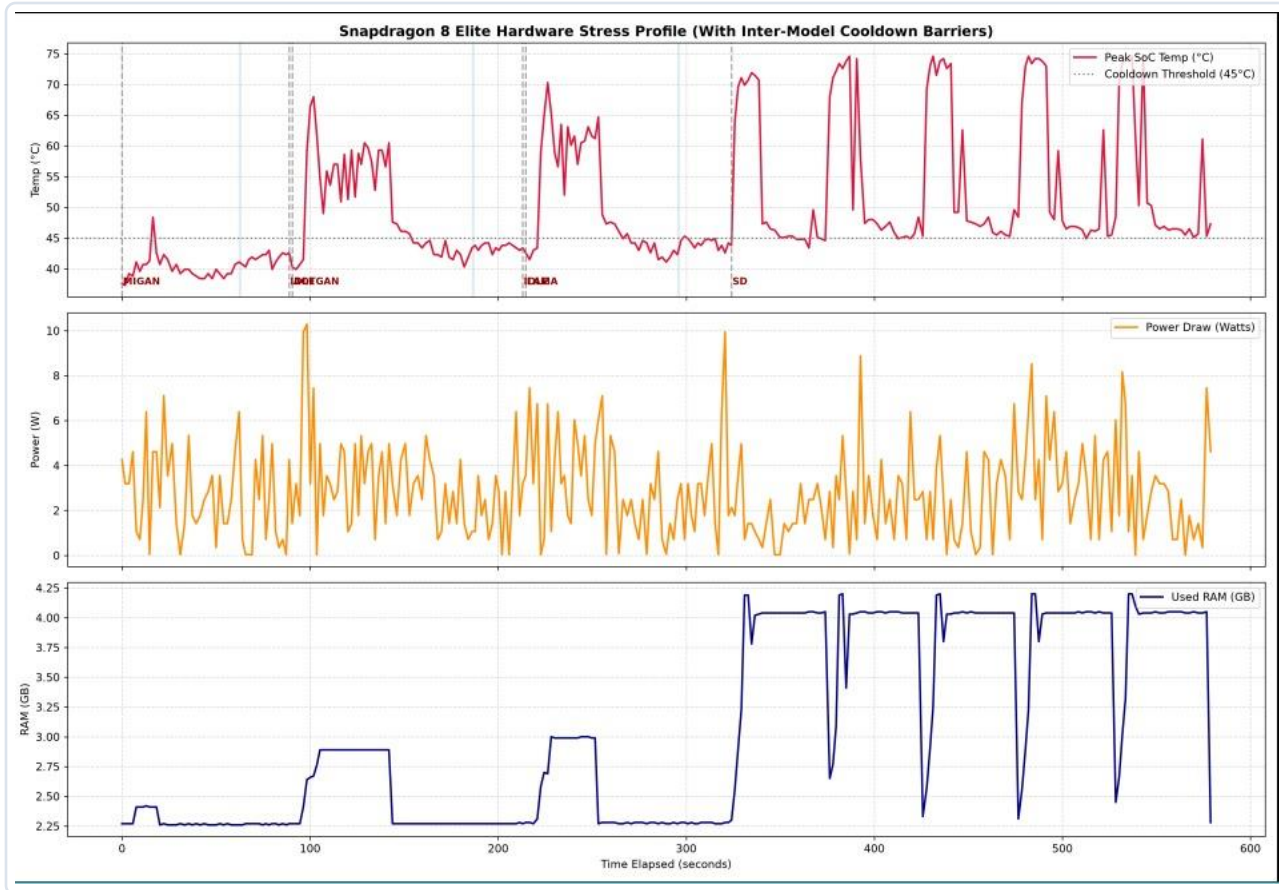
AOT-GAN — quality champion

Best hole-only reconstruction (20.86 dB) — sharpest hallucinated textures, at ~1 J/image.

LaMa-Dilated — balanced

Close to AOT-GAN on quality, most stable under extreme (30–50%) mask coverage.

What it actually costs the chip



Live SoC temperature, power draw, and RAM across a full back-to-back run of all four models, with enforced cooldown barriers between passes.



Qualcomm Profiler, live during an AOT-GAN run — NPU packet throughput spikes on every inference call.

A power bug we caught: Android sysfs reports current in μA and voltage in μV . Our first script read those raw, giving impossible 30–46 W readings. Fixed the unit conversion — realistic values now sit at 2–4 W.

Challenges we ran into (and how we handled them)

ISSUE	WHAT HAPPENED	HOW WE RESPONDED
Identical metrics across models	Evaluation runs reported the same numbers to 4 decimal places for different models — stale output files were being re-scored.	Isolated every model into its own output folder with strict {idx}_{model}.png naming; bug fully fixed.
Mask polarity inversion	A --normalise flag silently overwrote --reverse, feeding MIGAN un-inverted masks.	Standardized model-aware polarity handling (MIGAN uses inverse polarity vs. the other three); bug fully fixed.
HTP v79 driver mismatch	MIGAN/AOT-GAN failed DSP execution with QNN_DEVICE_ERROR_INVALID_CONFIG — an SNPE v2.47 vs v2.49 library mismatch, worsened by an Android 15 linker namespace collision.	Synced the full v2.49 HTP runtime stack across all models; all three GANs now run natively on the NPU.
No compiled MI-GAN model available	MI-GAN ships as PyTorch only — no ready .dlc from Qualcomm, unlike AOT-GAN and LaMa.	Exported to ONNX ourselves, then compiled and quantized to .dlc in-house.
Small dataset for benchmarking	Started with only 17 image pairs — nowhere near enough for reliable metrics.	Escalated to our faculty advisor and expanded to 200 pairs (Places365 + CelebA-HQ/FFHQ + PConv mask tiers).
Stable Diffusion's low PSNR/SSIM	SD scores far below the GANs on pixel-alignment metrics (10.1 dB vs ~28 dB).	Diagnosed, not a bug: diffusion hallucinates plausible new content and this checkpoint isn't inpainting-tuned yet — expected, not a failure (see previous slide).

What's next

01

Train a custom 9-input UNet for SD

The math is ready; training is next once evaluation is behind us. A purpose-built 9-channel input (noisy latent + masked-image latent + mask) should replace the current repaint-style trick and fix SD's inpainting-specific quality.

02

Model optimization pass

Complete the LaMa .dlc pipeline end-to-end, apply smarter per-channel quantization, and explore pruning / distillation from LaMa into a lighter student model.

03

Hardware optimization pass

Systematic HTP vs. GPU vs. CPU backend sweep for every model, runtime tuning (burst profiles, VTCM allocation), and delegate/init caching to cut per-image latency further.

04

Adaptive model selection

Our key differentiator: a learned router that picks the best model per image and per system state (thermal headroom, battery, mask size) instead of a single fixed choice.

Thank you

Questions welcome — code, models, and results are on GitHub Classroom.

github.com/ESW-M26/esw-m26-19_black_and_white